

Práctica 4

- Cruz Chavez Isaac

Ejemplo A Este ejercicio crea una variable inicializada en cero que guarda las sumas acumuladas y usa un bucle for para recorrer el arreglo. En cada paso toma el número que está en la posición i y se lo suma al guardado en total, después entrega el valor final acumulado.

Ejemplo B Crea una variable en cero que sirve como contador y utiliza un bucle for que toma cada número de la lista uno por uno. La condición evalúa si el número actual es mayor a la cifra límite, si es así le suma 1 al contador y al final devuelve el conteo de los números que cumplieron con la regla.

```
import java.util.Arrays;

public class P4Arreglos {

    // ===== EJEMPLO A (resuelto) =====
    // Suma todos los numeros de un arreglo y devuelve el total.
    // Este es el recorrido clasico: la variable i es la posicion.
    static int suma(int[] numeros) {
        int total = 0;
        for (int i = 0; i < numeros.length; i++) {
            total = total + numeros[i];
        }
        return total;
    }

    // ===== EJEMPLO B (resuelto) =====
    // Cuenta cuantos numeros son mayores que el umbral.
    // Este es el for-each: se usa cuando NO necesitas la posicion, solo el valor.
    static int cuantosPasan(int[] numeros, int umbral) {
        int cuantos = 0;
        for (int valor : numeros) {
            if (valor > umbral) {
                cuantos = cuantos + 1;
            }
        }
        return cuantos;
    }
}
```

Ejercicio 1

Este ejercicio solo crea y devuelve una lista ordenada con los cinco números enteros que se establecieron. Utiliza `new int[]` para fabricar el arreglo e inicializarlo directamente.

```
// ===== EJERCICIO 1 =====  
// Crea y devuelve un arreglo de int con estos cinco valores, en este orden:  
// 10, 20, 30, 40, 50. Aquí no hace falta ningún for.  
static int[] primerosCinco() {  
    return new int[]{ 10, 20, 30, 40, 50 };  
}
```

Ejercicio 2:

Calcula el promedio de una lista de números enteros y devuelve el resultado en decimales. Utiliza un bucle `for` para recorrer el arreglo y sumar cada uno de los elementos, mientras `double` es para que el resultado se pueda dar en decimales.

```
static double promedio(int[] numeros) {  
    // Reutilizamos el método suma() resuelto en el EJEMPLO A  
    return (double) suma(numeros) / numeros.length;  
}
```

Ejercicio 3.

Se asume que el primer número es el mayor y después utiliza un bucle `for` para recorrer el arreglo desde la segunda posición, si encuentra un número mayor al actual, actualiza la variable `max`. Al terminar de recorrer toda la lista, se devuelve el número más grande que haya encontrado.

```
static int maximo(int[] numeros) {  
    int max = numeros[0];  
    for (int i = 1; i < numeros.length; i++) {  
        if (numeros[i] > max) {  
            max = numeros[i];  
        }  
    }  
    return max;  
}
```

Ejercicio 4

Utiliza un bucle for para recorrer la lista y compara cada valor buscando al valor indicado, una vez que lo encuentra devuelve inmediatamente el valor. Si no lo encuentra devuelve -1.

```
static int posicionDe(int[] numeros, int buscado) { 1usage
    for (int i = 0; i < numeros.length; i++) {
        if (numeros[i] == buscado) {
            return i; // Regresa la primera posición donde coincida
        }
    }
    return -1;
}
```

Ejercicio 5.

Reserva memoria en un nuevo arreglo de la misma longitud y utiliza un ciclo for para recorrer el arreglo de izquierda a derecha, después asigna a la posición i del nuevo arreglo el elemento que está en la posición opuesta al arreglo original. Al final devuelve la nueva lista con los elementos invertidos.

```
static int[] invertido(int[] numeros) { 2usages
    int[] reves = new int[numeros.length];
    for (int i = 0; i < numeros.length; i++) {
        reves[i] = numeros[numeros.length - 1 - i];
    }
    return reves;
}
```

Ejercicio 6.

Guarda al primer alumno de la lista, con su nombre y calificación, asumiendo que es el mejor hasta el momento, después utiliza un ciclo for para recorrer el arreglo desde la segunda posición y accede a la propiedad de calificación de cada estudiante para compararla con la del “mejor hasta el momento” si encuentra una mayor calificación, reemplaza el objeto completo en la variable mejor con los datos de este estudiante. Al terminar de revisar la lista entera, devuelve solo el nombre del mejor.

```
static String nombreDelMasAlto(Alumno[] alumnos) { 1 usage
    Alumno mejor = alumnos[0];
    for (int i = 1; i < alumnos.length; i++) {
        if (alumnos[i].calificacion > mejor.calificacion) {
            mejor = alumnos[i];
        }
    }
    return mejor.nombre;
}
```

Ejercicio 7:

Se indica que la variable es una matriz, se recibe el número exacto de la fila que se quiere sumar y prepara una variable encargada de guardar el total de esa suma. Utiliza un ciclo for para recorrer las columnas de la fila seleccionada y va sumando los valores. Al final devuelve el valor total sumado.

```
static int sumaDeFila(int[][] matriz, int fila) { 1 usage
    int suma = 0;
    // matriz[fila].length nos da cuántas columnas tiene esa fila específica
    for (int i = 0; i < matriz[fila].length; i++) {
        suma += matriz[fila][i];
    }
    return suma;
}
```

Ejercicio 8:

Toma la frase completa, la divide cada que encuentra una coma y devuelve el arreglo donde mantienen sus espacios intactos, después inicializa una cadena vacía para construir la frase final. Utiliza un ciclo for para recorrer el arreglo y borrar los espacios en blanco a los lados, después evalúa si el nombre actual es el último elemento del arreglo y si no lo es, agrega un separador.

```
static String limpiaYJunta(String linea) { 1 usage
    String[] partes = linea.split(regex: ",");
    String resultado = "";

    for (int i = 0; i < partes.length; i++) {
        resultado += partes[i].trim();
        // Si no es el último elemento, le agregamos el separador
        if (i < partes.length - 1) {
            resultado += " | ";
        }
    }
    return resultado;
}
```